

# Cheat Sheet — OpenMP (Programação Concorrente)

Referência de consulta rápida. Organizado por: Regiões Paralelas → Worksharing → Escopo de Dados → Schedule → Sincronização → Tasks → Funções de Biblioteca / Variáveis de Ambiente.

## 1. Modelo Fork-Join

O programa começa com **1 thread (master)**. Ao encontrar `#pragma omp parallel`, o runtime cria um **time de threads** (fork) que executam o mesmo bloco de código. Ao final do bloco, há uma **barreira implícita**: todas esperam, e voltam a ser 1 thread só (join).

1 thread —[fork: parallel]→ N threads executam o bloco —[join]→ 1 thread

## 2. Diretivas de Região Paralela

| Diretiva                                   | O que faz                                                                                                             | Barreira implícita?  |
|--------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|----------------------|
| <code>#pragma omp parallel</code>          | Cria o time de threads; <b>todo</b> o bloco é replicado e executado por <b>cada</b> thread                            | Sim, no fim do bloco |
| <code>#pragma omp parallel for</code>      | Forma combinada: cria o time e distribui as iterações do laço seguinte entre as threads                               | Sim, no fim do laço  |
| <code>#pragma omp parallel sections</code> | Forma combinada de <code>parallel</code> + <code>sections</code> (distribui blocos de código distintos entre threads) | Sim                  |

Cláusulas comuns de `parallel`:

| Cláusula                                                                                 | Efeito                                                                                                                                   |
|------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| <code>num_threads(N)</code>                                                              | Define quantas threads o time terá (sobrepõe <code>OMP_NUM_THREADS</code> para essa região)                                              |
| <code>if(expr)</code>                                                                    | Se <code>expr</code> for falsa, a região roda sequencialmente (só a thread master)                                                       |
| <code>default(shared none)</code>                                                        | Define o escopo padrão de variáveis não listadas explicitamente. <code>none</code> obriga você a escopar tudo manualmente (boa prática!) |
| <code>shared(list)</code> / <code>private(list)</code> / <code>firstprivate(list)</code> | Ver seção 4                                                                                                                              |

### 3. Diretivas de Worksharing (divisão de trabalho)

Diretivas que **dividem** trabalho entre as threads de um time já criado (usadas dentro de `#pragma omp parallel`).

| Diretiva                                                             | O que faz                                                                                                                               | Barreira implícita?                        |
|----------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------|
| <code>#pragma omp for</code>                                         | Divide as iterações de um laço <code>for</code> entre as threads do time (cada iteração roda <b>uma vez só</b> , por <b>uma</b> thread) | Sim (a menos que use <code>nowait</code> ) |
| <code>#pragma omp single</code>                                      | Apenas <b>uma</b> thread do time (a primeira a chegar, não necessariamente a 0) executa o bloco                                         | Sim (a menos que use <code>nowait</code> ) |
| <code>#pragma omp master</code>                                      | Apenas a thread <b>0</b> (master) executa o bloco. As demais <b>não esperam</b> — seguem direto                                         | <b>Não</b>                                 |
| <code>#pragma omp sections</code> / <code>#pragma omp section</code> | Divide blocos de código <b>distintos</b> entre as threads (cada <code>section</code> roda em uma thread diferente)                      | Sim, no fim de <code>sections</code>       |

**Analogia:** `master` = "só o anfitrião compra o bolo, ninguém espera"; `single` = "quem chegar primeiro compra o bolo, todo mundo espera essa pessoa voltar".

Cláusulas de `for` / `parallel for`:

| Cláusula                                                                    | Efeito                                                                                                                                            |
|-----------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>schedule(tipo[, chunk])</code>                                        | Como as iterações são distribuídas — ver seção 5                                                                                                  |
| <code>collapse(n)</code>                                                    | Colapsa <code>(n)</code> laços aninhados em um único espaço de iteração antes de dividir entre threads (melhora balanceamento em laços aninhados) |
| <code>ordered</code>                                                        | Habilita o uso da construção <code>#pragma omp ordered</code> dentro do laço                                                                      |
| <code>nowait</code>                                                         | Remove a barreira implícita do fim do <code>for</code> / <code>single</code> / <code>sections</code>                                              |
| <code>reduction(op:var)</code>                                              | Ver seção 6                                                                                                                                       |
| <code>private</code> , <code>firstprivate</code> , <code>lastprivate</code> | Ver seção 4                                                                                                                                       |

## 4. Escopo de Dados (Data Scoping)

**Regra padrão:** variáveis declaradas **fora** da região paralela são `shared` por padrão — **exceto** a variável de controle de um `for` paralelizado, que é `private` automaticamente.

| Cláusula                          | Comportamento                                                                                                                                                                               |
|-----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>shared(list)</code>         | Todas as threads leem/escrevem na <b>mesma</b> posição de memória. Sem proteção = risco de condição de corrida (race condition)                                                             |
| <code>private(list)</code>        | Cada thread recebe uma cópia <b>própria e não-inicializada</b> . O valor de fora <b>não</b> é herdado, e ao sair da região, o valor original de fora permanece <b>indefinido/inalterado</b> |
| <code>firstprivate(list)</code>   | Como <code>private</code> , mas a cópia de cada thread é <b>inicializada com o valor que a variável tinha antes</b> da região começar                                                       |
| <code>lastprivate(list)</code>    | Como <code>private</code> , mas ao final do laço/seção, o valor da <b>última iteração/seção logicamente executada</b> é copiado de volta para a variável original                           |
| <code>default(shared none)</code> | Define o escopo padrão das variáveis não listadas. <code>none</code> força escopar tudo manualmente (evita erros silenciosos)                                                               |

Por que `shared` sem proteção é perigoso: uma linha como `resultado = resultado + tid` não é uma operação atômica — envolve **ler, somar e escrever** de volta na memória, em

passos separados. Se duas threads fazem isso ao mesmo tempo na mesma variável, uma pode sobrescrever o resultado da outra → **condição de corrida**, resultado imprevisível.

## 5. Cláusula `schedule` (Escalonamento de Laços)

| Tipo                          | Quando decide a divisão                        | Comportamento                                                                                                                                                                                                         | Melhor para                                                                         |
|-------------------------------|------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| <code>static[, chunk]</code>  | <b>Antes</b> de rodar (compile/início)         | Divide em blocos fixos e previsíveis. Sem <code>chunk</code> : blocos de tamanho $\approx N/\text{threads}$ . Com <code>chunk</code> : distribui em rodízio (round-robin) de <code>chunk</code> em <code>chunk</code> | Iterações com custo <b>uniforme</b> — menor overhead                                |
| <code>dynamic[, chunk]</code> | <b>Durante</b> a execução                      | Cada thread pega um pedaço (chunk, padrão=1); ao terminar, pede o próximo pedaço disponível na fila                                                                                                                   | Iterações com custo <b>desigual/imprevisível</b> — mas mais overhead de coordenação |
| <code>guided[, chunk]</code>  | Híbrido                                        | Começa com pedaços <b>grandes</b> , diminuindo progressivamente até o mínimo <code>chunk</code>                                                                                                                       | Meio-termo: carga desigual, mas com menos overhead que <code>dynamic</code> puro    |
| <code>auto</code>             | Implementação decide                           | Deixa o runtime/compilador escolher a melhor estratégia                                                                                                                                                               | Quando você não sabe qual usar                                                      |
| <code>runtime</code>          | Variável de ambiente <code>OMP_SCHEDULE</code> | Usa o que estiver definido em <code>OMP_SCHEDULE</code> em tempo de execução, sem recompilar                                                                                                                          | Testes/tuning sem recompilar                                                        |

⚠ **Importante:** `schedule` controla **quais** iterações cada thread executa, mas **nunca** garante a ordem de saída (ex: prints no terminal), a menos que se use `ordered`.

## 6. Sincronização

| Diretiva                                  | O que faz                                                                                                                                                                                                          |
|-------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>#pragma omp critical[(nome)]</code> | Só <b>uma thread por vez</b> pode executar o bloco (região de exclusão mútua). Se usar o mesmo <code>(nome)</code> em vários blocos, eles competem pelo mesmo "cadeado"; nomes diferentes = cadeados independentes |

| Diretiva                                                                                                                                                                                                                                                                                        | O que faz                                                                                                                                                                                                                                                                                                                                                                                                                    |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>#pragma omp atomic</code><br><code>[read write update capture]</code>                                                                                                                                                                                                                     | Protege <b>uma única operação de memória simples</b> (mais leve que <code>critical</code> ) <ul style="list-style-type: none"> <li><code>read</code>: leitura atômica</li> <li><code>write</code>: escrita atômica</li> <li><code>update</code> (padrão): operação tipo <code>x++</code>, <code>x += y</code></li> <li><code>capture</code>: lê e atualiza numa única operação atômica (ex: <code>z = x++</code>)</li> </ul> |
| <code>#pragma omp barrier</code>                                                                                                                                                                                                                                                                | Todas as threads do time esperam até que <b>todas</b> cheguem nesse ponto antes de continuar                                                                                                                                                                                                                                                                                                                                 |
| <code>#pragma omp ordered</code>                                                                                                                                                                                                                                                                | Dentro de um <code>for</code> com cláusula <code>ordered</code> , força que aquele trecho específico execute <b>na mesma ordem sequencial</b> do laço original                                                                                                                                                                                                                                                               |
| <code>nowait</code> (cláusula)                                                                                                                                                                                                                                                                  | Remove a barreira implícita do fim de <code>for</code> , <code>single</code> ou <code>sections</code> — usar com cuidado, pois pode gerar condições de corrida se as threads seguintes dependerem do resultado                                                                                                                                                                                                               |
| <code>reduction(operador:var)</code>                                                                                                                                                                                                                                                            | Cada thread mantém uma cópia privada da variável, acumula seu resultado parcial, e no final o runtime combina tudo com o <code>operador</code> (ex: <code>+</code> , <code>*</code> , <code>max</code> , <code>min</code> ) de forma segura, sem condição de corrida                                                                                                                                                         |
| <b><code>critical</code> vs <code>atomic</code></b> : use <code>atomic</code> quando a operação é simples (uma leitura/escrita/incremento de uma variável escalar) — é mais rápido. Use <code>critical</code> quando o bloco protegido tem <b>múltiplas instruções</b> ou lógica mais complexa. |                                                                                                                                                                                                                                                                                                                                                                                                                              |

## 7. Tasks (Tarefas)

Tasks são **unidades independentes de trabalho** (código + ambiente de dados + variáveis de controle), úteis quando o padrão de paralelismo não é um laço regular (ex: recursão, listas encadeadas, grafos de dependência).

### 7.1 Criação de tarefas

| Construção                             | Quando cria tarefa(s)                           |
|----------------------------------------|-------------------------------------------------|
| Início de região <code>parallel</code> | Cria tarefas <b>implícitas</b> (uma por thread) |
| <code>#pragma omp task</code>          | Cria <b>uma</b> tarefa explícita                |

| Construção                        | Quando cria tarefa(s)                                        |
|-----------------------------------|--------------------------------------------------------------|
| <code>#pragma omp taskloop</code> | Cria tarefas explícitas, uma para cada bloco (chunk) do laço |
| <code>#pragma omp target</code>   | Cria uma tarefa <b>target</b> (para dispositivos, ex: GPU)   |

O **runtime decide quando** as tarefas realmente executam — podem ser adiadas (colocadas num "pool de tarefas") ou executadas imediatamente. A thread que executa a tarefa pode ser **diferente** da thread que a criou.

## 7.2 Cláusulas de `#pragma omp task`

| Cláusula                                                                                 | Efeito                                                                                                                                                                            |
|------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>if(expr)</code>                                                                    | Se falsa, a tarefa é executada de forma <b>sequencial e imediata</b> (undeferred) pela thread que a encontrou                                                                     |
| <code>final(expr)</code>                                                                 | Se verdadeira, a tarefa não terá tarefas-filhas (força execução sequencial dos descendentes) — útil para "cortar" recursão excessivamente fina                                    |
| <code>mergeable</code>                                                                   | Permite que o ambiente de dados da tarefa seja <b>mesclado</b> com o da tarefa geradora (otimização)                                                                              |
| <code>untied</code>                                                                      | A tarefa pode ser retomada por <b>qualquer</b> thread do time (não precisa ser a que começou a executá-la). Padrão é <code>tied</code> (vinculada à mesma thread)                 |
| <code>priority(valor)</code>                                                             | Sugere prioridade de execução (valor numérico não-negativo; maior = recomendado executar antes)                                                                                   |
| <code>default(shared firstprivate none)</code>                                           | Escopo padrão das variáveis dentro da tarefa                                                                                                                                      |
| <code>private(list)</code> / <code>firstprivate(list)</code> / <code>shared(list)</code> | Mesmo conceito da seção 4, aplicado ao escopo da tarefa                                                                                                                           |
| <code>depend(in: list)</code>                                                            | Esta tarefa <b>depende</b> de tarefas anteriores que escreveram <code>out</code> / <code>inout</code> nessas variáveis                                                            |
| <code>depend(out: list)</code>                                                           | Esta tarefa <b>produz</b> dados nessas variáveis; tarefas futuras com <code>in</code> / <code>inout</code> sobre elas dependerão desta                                            |
| <code>depend(inout: list)</code>                                                         | Combina leitura e escrita — depende de qualquer tarefa anterior ( <code>in</code> , <code>out</code> ou <code>inout</code> ) sobre a variável, e futuras tarefas dependerão desta |

## 7.3 Regras de escopo padrão em tasks

| Situação                                                                                 | Escopo padrão                                                                                                 |
|------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| Variável estática ou global                                                              | <code>shared</code>                                                                                           |
| Variável local (automática) declarada <b>fora</b> da task                                | <code>firstprivate</code> (se a task for "órfã", sem task-pai)                                                |
| Variável local declarada <b>dentro</b> do bloco <code>task</code>                        | <code>private</code>                                                                                          |
| Variável <code>static</code> declarada dentro do bloco <code>task</code>                 | <code>shared</code>                                                                                           |
| Objeto alocado dinamicamente ( <code>malloc</code> , <code>new</code> ) apontado de fora | <code>shared</code> (o ponteiro pode ser <code>firstprivate</code> , mas o conteúdo apontado é compartilhado) |
| Task com task-pai                                                                        | Herda os atributos de compartilhamento da tarefa pai                                                          |

## 7.4 Sincronização de tarefas

| Diretiva                                                                  | O que faz                                                                                                                                                                                                   |
|---------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>#pragma omp taskwait</code>                                         | A tarefa atual espera suas <b>tarefas-filhas diretas</b> terminarem (não espera netos/descendentes mais profundos)                                                                                          |
| <code>#pragma omp taskgroup</code>                                        | Espera <b>todas as tarefas descendentes</b> (filhas e seus descendentes) dentro do bloco terminarem — sincronização mais profunda que <code>taskwait</code> . Pode ter cláusula <code>task_reduction</code> |
| <code>#pragma omp taskyield</code>                                        | Ponto explícito onde a tarefa atual pode ser suspensa, liberando a thread para executar outra tarefa (evita bloqueios)                                                                                      |
| Barreira implícita ( <code>barrier</code> ), fim de <code>parallel</code> | Garante que <b>todas</b> as tarefas criadas por qualquer thread do time terminem                                                                                                                            |

## 7.5 `taskloop` — paralelizar laços via tasks

```
c
#pragma omp taskloop [simd] [cláusulas]
for (...) { ... }
```

| Cláusula                                                                                                                                                                                                                                                                    | Efeito                                                                                                                 |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| <code>grainsize(n)</code>                                                                                                                                                                                                                                                   | Cada bloco (chunk) de tarefa terá entre <code>n</code> e <code>2n</code> iterações                                     |
| <code>num_tasks(n)</code>                                                                                                                                                                                                                                                   | Cria exatamente <code>n</code> tarefas para cobrir todas as iterações                                                  |
| <code>nogroup</code>                                                                                                                                                                                                                                                        | Por padrão, <code>taskloop</code> cria uma região <code>taskgroup</code> implícita; essa cláusula remove essa barreira |
| <code>collapse(n)</code>                                                                                                                                                                                                                                                    | Mesmo conceito da seção 3, aplicado a laços aninhados dentro do <code>taskloop</code>                                  |
| Também aceita: <code>shared</code> , <code>private</code> , <code>firstprivate</code> , <code>lastprivate</code> , <code>default</code> , <code>final</code> , <code>if</code> , <code>untied</code> , <code>mergeable</code> , <code>depend</code> , <code>priority</code> | Mesmo significado das seções 4 e 7.2                                                                                   |
| <code>taskloop simd</code> : além de dividir em tarefas, aplica vetorização SIMD dentro de cada bloco.                                                                                                                                                                      |                                                                                                                        |

## 8. Funções de Biblioteca (Runtime API)

| Função                                 | O que retorna/faz                                                                                                  |
|----------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| <code>omp_get_num_threads()</code>     | Número de threads <b>no time atual</b> (só é significativo dentro de uma região paralela)                          |
| <code>omp_get_thread_num()</code>      | O <code>id</code> (tid) da thread que chama a função, de 0 a N-1                                                   |
| <code>omp_get_num_procs()</code>       | Número de processadores/núcleos disponíveis na máquina                                                             |
| <code>omp_in_parallel()</code>         | Retorna se o código está atualmente dentro de uma região paralela ativa                                            |
| <code>omp_get_wtime()</code>           | Retorna o tempo (em segundos, <code>double</code> ) desde uma referência fixa — usado para medir tempo de execução |
| <code>omp_set_num_threads(n)</code>    | Define o número de threads a usar nas próximas regiões paralelas                                                   |
| <code>omp_init_lock(&amp;lock)</code>  | Inicializa uma trava (lock) manual                                                                                 |
| <code>omp_set_lock(&amp;lock)</code>   | Adquire a trava (bloqueia se outra thread já a possui)                                                             |
| <code>omp_unset_lock(&amp;lock)</code> | Libera a trava                                                                                                     |



---

## 9. Variáveis de Ambiente

| Variável                     | Efeito                                                                                                          |
|------------------------------|-----------------------------------------------------------------------------------------------------------------|
| <code>OMP_NUM_THREADS</code> | Define o número padrão de threads (sobrepota por <code>num_threads()</code> no código)                          |
| <code>OMP_SCHEDULE</code>    | Define o tipo/chunk de <code>schedule</code> usado quando a cláusula do código é <code>schedule(runtime)</code> |
| <code>OMP_STACKSIZE</code>   | Tamanho da pilha de cada thread                                                                                 |
| <code>OMP_PLACES</code>      | Define o mapeamento de threads para localizações físicas (núcleos, sockets)                                     |
| <code>OMP_PROC_BIND</code>   | Controla se/como as threads ficam "fixadas" a processadores específicos                                         |

---

## 10. Compilação

```
bash
```

```
gcc -fopenmp arquivo.c -o programa # habilita suporte a OpenMP no GCC
./programa # executa
OMP_NUM_THREADS=4 ./programa # define threads via variável de ambiente
```

---

## 11. Resumo mental rápido

- `parallel` → cria threads. `for` → divide iterações. `parallel for` → os dois juntos.
- `single` → uma thread qualquer, com barreira. `master` → sempre a thread 0, sem barreira.
- `shared` → risco de condição de corrida. `private` → cópia isolada, não inicializada. `firstprivate` → cópia isolada, inicializada com valor de fora.
- `static` → previsível, baixo overhead. `dynamic` → adaptável, mais overhead. `guided` → híbrido.
- `atomic` → protege 1 operação simples. `critical` → protege um bloco inteiro (mais caro).
- `task` → unidade de trabalho independente, útil fora do padrão de laço regular (recursão, listas, grafos de dependência via `depend`).
- `taskwait` → espera só filhos diretos. `taskgroup` → espera toda a descendência.

